

Alumni Portal — SQL Query Practice Sheet (pgAdmin)

Same idea as the project-management endpoint sheet you pasted: each line is an "API" you'd have to serve with a query. Write the SQL yourself in pgAdmin against the schema + seed data from the two migrations. No answers included on purpose — that's the practice. Difficulty tags: [E] easy, [M] medium, [H] hard (window functions / CTEs / transactions).

ALUMNI

- [E] GET /api/alumni — list all alumni
- [E] GET /api/alumni/:id — single alumni by id
- [E] GET /api/alumni?campus=camp_jal — filter by campus
- [M] GET /api/alumni?branch=Computer Science and Engineering&batchYear=2022 — multi-field filter
- [M] GET /api/alumni/search?q=riya — case-insensitive partial match on name/email
- [E] POST /api/alumni — insert a new alumni row
- [E] PATCH /api/alumni/:id — update current_role/current_company
- [M] DELETE /api/alumni/:id — delete + verify cascade behavior on education/work_experiences/posts
- [H] GET /api/alumni/duplicates — find alumni sharing the same email domain or phone (self-join / GROUP BY HAVING COUNT > 1)
- [H] GET /api/alumni/:id/full-profile — one query joining alumni + education + work_experiences + startups (LEFT JOIN, avoid row-count blowup issues)

ACADEMIC NORMALIZATION

- [E] GET /api/academic-options?type=BRANCH — active canonical branches
- [M] GET /api/alumni/needs-review — all alumni flagged needs_review = true
- [M] GET /api/normalization/variants?field=branch — DISTINCT raw branch strings not present in academic_options.value (anti-join / NOT IN / NOT EXISTS)
- [H] POST /api/normalization/merge — transaction: UPDATE alumni + UPDATE registration_requests + INSERT normalization_merge_logs, all-or-nothing
- [M] GET /api/normalization/logs?field=branch — audit history for one field
- [H] GET /api/academic-options/usage-count — for each canonical branch, COUNT(*) of alumni currently using it (LEFT JOIN + GROUP BY, including zero-usage options)

REGISTRATION REQUESTS (staging)

- [E] GET /api/registration-requests?status=PENDING
- [M] GET /api/registration-requests?campus=camp_jal&needsReview=true
- [H] POST /api/registration-requests/:id/approve — transaction: INSERT into alumni from the request row, UPDATE registration_requests.status = APPROVED
- [M] GET /api/registration-requests/stale — requests older than 30 days still PENDING (date arithmetic: NOW() - INTERVAL)

EVENTS & RSVPS

- [E] GET /api/events — published events, soonest first
- [E] GET /api/events/:id/rsvps — all RSVPs for one event
- [M] GET /api/events/:id/rsvp-summary — COUNT of ATTENDING / MAYBE / NOT_ATTENDING per event (GROUP BY status)
- [M] GET /api/alumni/:id/events — events an alumni RSVP'd to (JOIN rsvps → events)
- [H] GET /api/events/low-turnout — events where ATTENDING count < 50% of total RSVPs (subquery/CTE + ratio)
- [E] POST /api/events/:id/rsvp — upsert an RSVP (respect the alumni_id+event_id UNIQUE constraint — use ON CONFLICT)

JOBS

- [E] GET /api/jobs?category=VACANCY&isActive=true
- [M] GET /api/jobs/by-company — GROUP BY company, COUNT(*) active postings
- [M] GET /api/alumni/:id/jobs-posted — jobs posted by one alumni
- [H] GET /api/jobs/expiring-soon — expire_at within next 7 days, ordered ascending (date filtering + ORDER BY)

SOCIAL GRAPH (connections & follows)

- [E] GET /api/alumni/:id/connections?status=ACCEPTED
- [M] GET /api/alumni/:id/pending-requests — connections where receiver_id = :id AND status = PENDING
- [M] GET /api/alumni/:id/followers — join alumni_follows → alumni on follower_id
- [M] GET /api/alumni/:id/following — join alumni_follows → alumni on following_id
- [H] GET /api/alumni/mutual-connections?a=al_riya&b=al_arjun — alumni who follow both A and B (self-join alumni_follows twice, or INTERSECT)
- [H] GET /api/alumni/suggested-connections/:id — alumni followed by people :id follows, excluding people :id already follows (2-hop graph query)

NOTIFICATIONS

- [E] GET /api/alumni/:id/notifications — via notification_states JOIN notifications
- [E] GET /api/alumni/:id/notifications/unread-count — COUNT WHERE is_read = false
- [M] PATCH /api/alumni/:id/notifications/mark-all-read — bulk UPDATE notification_states
- [H] GET /api/notifications/campaign/:campaignGroupId/progress — sent_count/total_targets, plus % complete (computed column)
- [H] GET /api/notifications/keyset — SELECT * FROM alumni WHERE id > :cursor ORDER BY id ASC LIMIT 500 (practice the exact keyset pagination pattern from the campaign worker)

SEARCH / REPORTING (the "GET /api/search/..." pattern from your screenshot)

- [E] GET /api/search/alumni?q=query — ILIKE across name/email/current_company
- [M] GET /api/search/jobs?q=query — ILIKE across title/company/description

[H] GET /api/reports/alumni-by-batch-branch — pivot-style report: rows = batch_year, columns = branch, cell = COUNT(*) (CASE WHEN + GROUP BY, or crosstab)

[H] GET /api/reports/campus-engagement — per campus: alumni count, % registered, avg followers_count, event RSVP count (multiple JOINS + subqueries in SELECT list)

[H] GET /api/reports/top-alumni-by-engagement — RANK() alumni by (followers_count + post count + connection count) using a window function

Suggested grind order

Alumni + Events (plain SELECT, WHERE, JOIN, GROUP BY)

Academic Normalization + Registration Requests (anti-joins, transactions, ON CONFLICT)

Social Graph (self-joins, multi-hop graph queries)

Notifications + Reporting (window functions, CTEs, keyset pagination)

Once you can write all of these cold, you're basically ready to explain and defend query choices in a system-design interview — which is the actual goal here, not the SQL syntax itself.